FACULTY OF ARTS

KU LEUVEN

Scripting languages

Lecture 7: Text processing, errors & exceptions

Tim Van de Cruys



Wednesday 4 November 2026

Text processing

- Up till now, we used basic Python machinery to manipulate text
- Python `str` object, with different methods:
  - `split`, `join`
  - `lower`, `strip`
  - `isalpha`, `isnumeric`
- e.g. `doc.split()` to interpret a string as a list of words
- But this only works up to a certain extent

Splitting on whitespace

```
s = "Mr. J. Simpson said: 'Astounding!'"
s.split()
```

```
["Mr.", "J.", "Simpson", "said:", "'Astounding!'"]
```

- Punctuation sticks to words: `"said:"` instead of `"said"` and `":"`
- Quotes are not separated: `"'Astounding!'"` is one token
- No sentence boundaries detected

Preprocessing

- Preprocessing of raw text is an important step
  - word segmentation (**tokenization**)
  - **sentence detection**
  - **normalization** (e.g. lemmatization)
- These steps convert raw text into a structured representation
- Necessary before any kind of text analysis

Tokenization

- Text is encoded as a string of characters
- Words, spaces, and punctuation is encoded in the same way
- We need to define and mark word boundaries: **tokenization**

Tokenization

- task: splitting a string up into tokens
- tokens are (1) words, (2) punctuation marks, (3) various other things
- first and most basic step in text analysis, should be done automatically
- various algorithms and tools exist, none is perfect
- Where to split text? Most important cues: whitespace, punctuation

```
>>> tokenize('"Hark! A vagrant," he said.')
['"', 'Hark', '!', 'A', 'vagrant', ',', '"', 'he', 'said', '.']
```

Tricky tokenization cases

One or two tokens?

- prime minister
- New York
- I'm

Tricky tokenization cases

One or two tokens?

- prime minister
- New York
- I'm

More tricky cases:

```
John's bike
New York-based
nineteenth- and twentieth-century writers
(SAM)-missile launchers
i.e.  Ph.D. &c.
;-) :)))) #yolo#lit#hashtag
https://en.wikipedia.org/wiki/Bicycle
8-Cyclopentyl-1,3-dimethylxanthine
```

Sentence detection

- Certain punctuation characters are unambiguous (‘?’, ‘!’)
- A dot (‘.’) is very ambiguous
  - *Obama held his final address to the U.N. General Assembly on Wednesday.*
  - *Different genres such as the motet, the cantata, etc. developed throughout the centuries.*
- Context is important to determine where a sentence ends

Normalization

Word forms

Should we count these separately:

`walk, walks, walked, walking, …`

- Recognizing that these are instances of the same word is called **lemmatization**
- Requires language-specific tools
- Most work does not apply this step for simplicity, and forms may contain useful distinctions

Spacy & NLTK

- Two different libraries for **natural language processing** (NLP)
- Useful if you're working with text
- Both libraries contain preprocessing pipelines (tokenization, sentence detection, lemmatization)
- Can equally be used for:
  - Linguistic annotation
  - Keyword detection, word-based statistics

NLTK

Tokenization

```
import nltk
s = "Mr. J. Simpson said: 'Astounding!'"
```

```
nltk.word_tokenize(s)
```

```
['Mr.', 'J.', 'Simpson', 'said', ':', "'Astounding", '!', "'"]
```

NLTK

Tokenization

```
import nltk
s = "Mr. J. Simpson said: 'Astounding!'"
```

```
nltk.word_tokenize(s)
```

```
['Mr.', 'J.', 'Simpson', 'said', ':', "'Astounding", '!', "'"]
```

```
nltk.sent_tokenize("Mr. J. Simpson said so. Astounding.")
```

```
['Mr. J. Simpson said so.', 'Astounding.']
```

- Compare to `split()`: punctuation is now separated from words
- Abbreviations like `Mr.` and `J.` are handled correctly

NLTK

Lemmatization

```
from nltk.stem import WordNetLemmatizer
lemmatizer = WordNetLemmatizer()
print("rocks :", lemmatizer.lemmatize("rocks"))
print("corpora :", lemmatizer.lemmatize("corpora"))
# a denotes adjective in "pos"
print("better :", lemmatizer.lemmatize("better", pos="a"))
```

```
rocks : rock
corpora : corpus
better : good
```

NLTK: explore keywords in context

NLTK `Text` object offers:

- Concordance
- Similar words
- Common contexts
- Collocations

```
import nltk
with open('melville-mobydick.txt', 'rt') as in_file:
    moby_dick_tokens = nltk.word_tokenize(in_file.read())
moby_dick = nltk.Text(moby_dick_tokens)
with open('austen-sense.txt', 'rt') as in_file:
    austen_sense_tokens = nltk.word_tokenize(in_file.read())
austen_sense = nltk.Text(austen_sense_tokens)
```

Concordance

```
moby_dick.concordance('monstrous')
```

```
Ho ' s Story . CHAPTER 55 . Of the Monstrous Pictures of Whales . CHAPTER 56 .
ong the former , one was of a most monstrous size ... . This came towards us ,
n of the Psalms_ . " Touching that monstrous bulk of the whale or ork we have r
ll over with a heathenish array of monstrous clubs and spears . Some were thick
d as you gazed , and wondered what monstrous cannibal and savage could ever hav
that has survived the flood ; most monstrous and most mountainous ! That Himmal
they might scout at Moby Dick as a monstrous fable , or still worse and more de
of Radney. ' " CHAPTER 55 . Of the Monstrous Pictures of Whales . I shall ere l
ing Scenes . In connexion with the monstrous pictures of whales , I am strongly
ere to enter upon those still more monstrous stories of them which are to be fo
ght have been rummaged out of this monstrous cabinet there is no telling . But
e of Whale-Bones ; for Whales of a monstrous size are oftentimes cast up dead u
```

- A concordance shows every occurrence of a word in its context
- Also called KWIC: Key Word In Context

Similar words

```
moby_dick.similar("monstrous")
```

```
mean part maddens doleful gamesome subtly uncommon careful untoward
exasperate loving passing mouldy christian few true mystifying
imperial modifies contemptible
```

```
austen_sense.similar("monstrous")
```

```
very heartily so exceedingly remarkably as vast a great amazingly
extremely good sweet
```

Austen uses *monstrous* as an adverb of degree, with a positive meaning (Phillips 1969, p. 329)

Phillips (1969). Jane Austen's English. https://www.jstor.org/stable/43342482

Common contexts

```
austen_sense.common_contexts(['monstrous', 'very'])
```

```
a_pretty am_glad a_lucky is_pretty, be_glad
```

Common contexts

```
austen_sense.common_contexts(['monstrous', 'very'])
```

```
a_pretty am_glad a_lucky is_pretty, be_glad
```

i.e., Sense & Sensibility contains both “a very pretty” and “a monstrous pretty” etc.

```
austen_sense.concordance('monstrous')
```

```
Displaying 11 of 11 matches:
 `` Now , Palmer , you shall see a monstrous pretty girl . '' He immediately we
your sister is to marry him . I am monstrous glad of it , for then I shall have
[...]
```

Collocations

A **collocation** is a sequence of words that occur together more frequently than expected (based on their frequency). As an example, *red wine* is a collocation, but *the wine* is not.

- *red* and *wine* appear together much more often than their individual frequencies would predict
- *the* and *wine* appear together often; but *the* appears with *everything*, so this is not surprising
- The key idea: compare **observed** co-occurrence to **expected** co-occurrence

Collocations: examples

```
moby_dick.collocations()
```

```
Sperm Whale; Moby Dick; White Whale; old man; Right Whale; Captain
Ahab; sperm whale; New Bedford; Captain Peleg; Mr. Starbuck; Cape
Horn; cried Ahab; Mrs. Hussey; years ago; chief mate; lower jaw;
Father Mapple; white whale; ivory leg; cried Stubb
```

```
austen_sense.collocations()
```

```
Mrs. Jennings; Colonel Brandon; Sir John; Lady Middleton; Mrs.
Dashwood; Miss Dashwood; Mrs. Ferrars; every thing; thousand pounds;
dare say; Mr. Palmer; Miss Steeles; said Elinor; Mrs. Palmer; Miss
Steele; every body; Mr. Willoughby; John Dashwood; great deal; Harley Street
```

- Character names and titles dominate; these are strong collocations in novels
- Collocation measures are explored in the practical session

Spacy

```
import spacy
nlp = spacy.load("en_core_web_sm")
s = "Mr. J. Simpson said: 'Astounding!'"
doc = nlp(s)
```

- Spacy processes the entire text at once through a **pipeline**
- The result is a `Doc` object containing all annotations
- Tokenization, sentence detection, lemmatization, part-of-speech tagging, dependency parsing: all in one call

Spacy: tokenization and sentences

```
for token in doc:
    print(token.text)
```

```
Mr.  J.  Simpson  said  :  '  Astounding  !  '
```

Spacy: tokenization and sentences

```
for token in doc:
    print(token.text)
```

```
Mr.  J.  Simpson  said  :  '  Astounding  !  '
```

```
s2 = "Mr. J. Simpson said so. Astounding."
doc2 = nlp(s2)
for sent in doc2.sents:
    print(sent)
```

```
Mr. J. Simpson said so.
Astounding.
```

- Like NLTK, Spacy correctly handles abbreviations

Spacy: linguistic annotations

```
for token in doc:
    print(token.text, token.lemma_,
            token.pos_, token.is_stop)
```

```
Mr.          Mr.          PROPN   False
J.           J.           PROPN   False
Simpson      Simpson      PROPN   False
said         say          VERB    False
:            :            PUNCT   False
'            '            PUNCT   False
Astounding   astounding    ADJ     False
!            !            PUNCT   False
'            '            PUNCT   False
```

- `token.lemma_`: the base form; `token.pos_`: part of speech (`PROPN` proper noun, `VERB`, `ADJ`, `PUNCT`, ...); `token.is_stop`: stop word (*the*, *is*, *of*)
- Also available: `token.dep_` (syntactic dependency), `token.shape_` ("Simpson" → "Xxxxx")

A class for a text

- Last week's `Text` class stored `raw.split()` in `self.words`; two changes: the words come from `nltk.word_tokenize`, and the sentences are stored as well

```
import nltk
from collections import Counter

class Text:
    def __init__(self, filename):
        self.filename = filename
        with open(filename) as in_file:
            raw = in_file.read()
        self.words = [token.lower() for token in nltk.word_tokenize(raw)]
        self.sentences = nltk.sent_tokenize(raw)

    def word_count(self):
        return len(self.words)

    def most_common(self, n):
        return Counter(self.words).most_common(n)
```

Using the class

```
In: emma = Text('books/austen-emma.txt')
In: alice = Text('books/carroll-alice.txt')
In: emma.word_count(), alice.word_count()
Out: (191855, 33535)
In: len(alice.sentences)
Out: 1625
In: alice.most_common(3)
Out: [(',', 2418), ('the', 1616), ("'", 1311)]
```

- Nothing else changed: `word_count()` and the code that uses the class stay the same, only the tokenization got better
- Punctuation is still counted: filtering it out is one more line in the constructor (practical session)

A corpus of texts

- Composition: a `Corpus` *has* a list of `Text` objects

```
from glob import glob

class Corpus:

    def __init__(self, pattern):
        self.texts = []
        for filename in sorted(glob(pattern)):
            self.texts.append(Text(filename))

    def word_count(self):
        total = 0
        for text in self.texts:
            total += text.word_count()
        return total

books = Corpus('books/*.txt')
books.word_count()
```

Errors

- Programs often fail in unexpected ways (wrong input, missing file, division by zero)
- You have already seen some:
  - `FileNotFoundError`: file doesn't exist
  - `UnicodeDecodeError`: wrong text encoding
  - `KeyError`: key not in dictionary
- Python calls these **exceptions**
- By default, an exception stops the program
- Sometimes we want to **handle** the error and continue

Types of exceptions

SyntaxError	unable to parse code (missing comma, colon, parenthesis?)
NameError	non-existent variable/function name (typo?)
IndexError	access element beyond end of sequence `seq[n] with n >= len(seq)`
KeyError	key not in dictionary
AttributeError	`object.attribute`, but object does not have attribute
TypeError	mixing up str/int/etc.
ValueError	generic error: right type, wrong value
FileNotFoundError	check filename / path
UnicodeDecodeError	file is not in the encoding you expect

This is not an exhaustive list; see
`https://docs.python.org/3/library/exceptions.html`

Tracebacks

- **Traceback**: the chain of function calls that led to an error

```
>>> sent = "The quick brown fox jumped."
>>> words = sent.split()
>>> print(words.lower())
Traceback (most recent call last):
  <ipython-input-11-625626ddc39c> in <module>
AttributeError: 'list' object has no attribute 'lower'
>>>
```

Tracebacks

- **Traceback**: the chain of function calls that led to an error

```
>>> from os.path import splitext
>>> splitext(['not', 'a', 'filename'])
Traceback (most recent call last):
  <ipython-input-11-625626ddc39c> in <module>
  File "/usr/lib/python3.7/posixpath.py", line 122, in splitext
    p = os.fspath(p)
TypeError: expected str, bytes or os.PathLike object, not list
>>>
```

- Read from the bottom up: the last line says **what** went wrong, the lines above say **where**
- The cause of the error will probably be somewhere in your own code: look at the last line that refers to it
- If error message is unclear: check the internet

Debugging: general strategy

Think carefully about your code, and add `print` statements to determine what is happening in intermediate steps

Debugging: general strategy

Think carefully about your code, and add `print` statements to determine what is happening in intermediate steps

Go through each line of code

- Think like a computer: what is happening? Is it what you intended?
- Ask yourself: what is the input and output? Confirm: `print` values
- Does the problem only occur with particular values? What is special about them?
- Difficult problems require a lot of going back and forth

Exception handling

- You can avoid running into exceptions by using appropriate if-statements in your code
- Sometimes you cannot foresee all potential exceptions your program may run into
- Python has a mechanism to 'catch' exceptions

```
In : try:
...:     result = i / j
...: except ZeroDivisionError:
...:     print('cannot divide by zero')
...: else:
...:     print('result is', result)
...:
```

Exception handling

- You can avoid running into exceptions by using appropriate if-statements in your code
- Sometimes you cannot foresee all potential exceptions your program may run into
- Python has a mechanism to 'catch' exceptions

```
In : i = 10
In : j = 0
In : try:
...:     result = i / j
...: except ZeroDivisionError:
...:     print('cannot divide by zero')
...: else:
...:     print('result is', result)
...:
Out: cannot divide by zero
```

Exception handling

- You can avoid running into exceptions by using appropriate if-statements in your code
- Sometimes you cannot foresee all potential exceptions your program may run into
- Python has a mechanism to 'catch' exceptions

```
In : i = 10
In : j = 1
In : try:
...:     result = i / j
...: except ZeroDivisionError:
...:     print('cannot divide by zero')
...: else:
...:     print('result is', result)
...:
Out: result is 10.0
```

Try / except: a practical example

Processing multiple files, some of which might have encoding issues:

```
from glob import glob
for filename in glob('data/*.txt'):
    try:
        with open(filename) as in_file:
            text = in_file.read()
            print(f'{filename}: {len(text)} chars')
    except UnicodeDecodeError:
        print(f'{filename}: encoding error, skipped')
```

- Without `try`/`except`: one bad file stops the entire process
- With `try`/`except`: we skip the problem and continue
- Catch **specific** exceptions; `except Exception` catches everything and can hide real bugs

Try it: catching an error

- The list below mixes numbers and words. Convert the numbers to integers and skip the rest, without an if-statement

```
values = ['12', 'twelve', '7', '3.5', '40']
```

Practical session / reading

- Practical session 6
- NLTK book, chapter 1: `https://www.nltk.org/book/ch01.html`
- The code-along notebook of this lecture is on Toledo: re-run the examples at home